

c0a25046bf / ProjExD_Group14

🔍

👤

+

🕒

🔗

📄

📧

<> Code

🕒 Issues

🔗 Pull requests

🔗 Agents

🕒 Actions

📁 Projects

📖 Wiki

🛡 Security and quality

📈 Insights

📁

🔗 main

ProjExD_Group14 / README.md

📄

🔍

Go to file

T

⋮

c0a25046bf

readme最終版

74e85d0 · 2 minutes ago

🕒

36 lines (29 loc) · 1.33 KB

PreviewCodeBlame

🔄

👤

Raw

📄

📥

✎

⌵

⋮

Koukaton Up

実行環境の必要条件

- python >= 3.10
- pygame >= 2.1

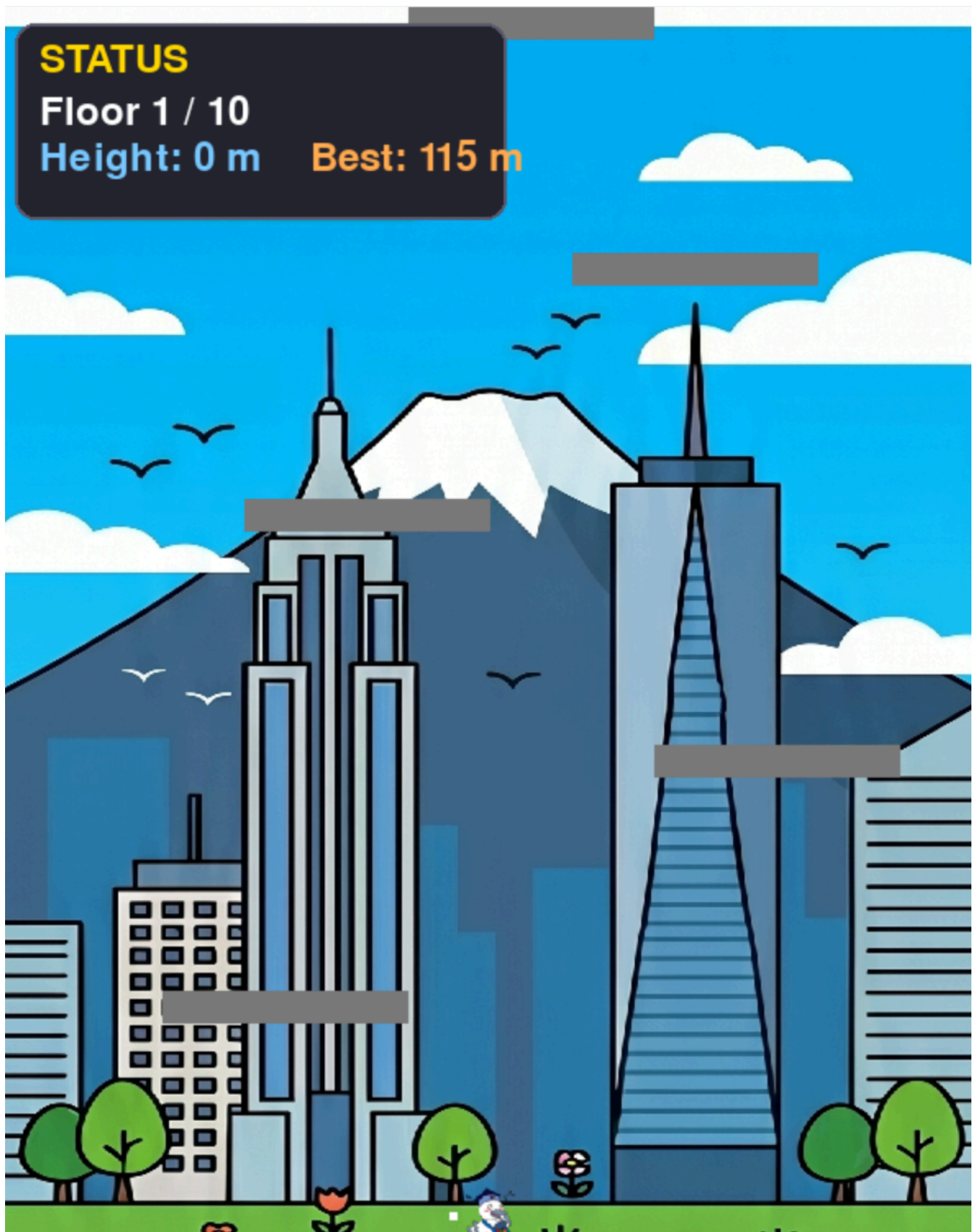
ゲームの概要

- 主人公(工科トン)をどんどん上に登らせるゲーム
- 参考URL：[Only Up!](#)：[Square On Top: Jump](#)

ゲームの遊び方

- キーを使って床をどんどん登って上に行く
- 落ちたら床があるところまで落ちる
- 面白い床もあるよ!

ゲームの実装



共通基本機能

- 矢印でジャンプの方向決めてスペースの押す長さでジャンプ力を決める
- 壁に当たると跳ね返る
- ゴールを目指して頑張ろう！

分担追加機能

- 1_グラフィック（画像）変更[ゆら],
- 2_当たり判定の厳密化（頭ぶつけ・すり抜け防止）[うりゅう],
- 3_ステージギミック追加[そら],
- 4_音楽[こうへい],
- 5_左上のUI（高さ・情報の表示）[こな]

Todo

- タイマーを付けたかった

メモ

- クラス内の変数は、すべて、「get_変数名」という名前のメソッドを介してアクセスするように設計してある
- すべてのクラスに関係する関数は、クラスの外で定義してある

 c0a25046bf / ProjExD_Group14

 | 

 Code  Issues  Pull requests  Agents  Actions  Projects  Wiki  Security and quality  Insights

  main  ProjExD_Group14 / koukaton_up.py 

 Go to file  T 

 c0a25046bf クリア可能des cd9216f · 32 minutes ago 

563 lines (458 loc) · 21.9 KB

Code Blame

 Raw    

```
1  import pygame
2  import sys
3  import os
4  import random
5
6  # カレントディレクトリの固定
7  os.chdir(os.path.dirname(os.path.abspath(__file__)))
8
9  # =====
10 # 設計方針・共通ルール
11 # 1. クラス内の変数は、すべて「get_変数名」というメソッドを介してアクセスする
12 # 2. クラスに関係するすべての操作関数は、クラスの外部で定義する
13 # =====
14
15 # 画面・物理設定
16 WIDTH, HEIGHT = 600, 800
17 FPS = 60
18 GRAVITY = 0.6
19
20 # カラー定義
21 BLACK = (20, 20, 25)
22 BLUE = (100, 150, 255)
23 WHITE = (255, 255, 255)
24 GOLD = (255, 215, 0)
25
26 DARK_PANEL = (35, 35, 45)
27 PANEL_BORDER = (90, 90, 110)
28 LIGHT_BLUE = (120, 200, 255)          # ステータスUI用 (青)
29 ORANGE = (255, 170, 80)              # ベスト記録UI用 (オレンジ)
30
31 # ギミック床の色定義
32 COLOR_NORMAL = (120, 120, 120)      # 通常の床 (灰色)
33 COLOR_FAKE = (180, 0, 180)          # すり抜ける床 (紫)
34 COLOR_ICE = (100, 200, 255)         # 滑る床 (水色)
35 COLOR_TRAMPOLINE = (255, 140, 0)    # 大ジャンプする床 (オレンジ)
36 COLOR_TRAP = (255, 50, 50)          # 最初に戻る床 (赤)
37 COLOR_RED_UI = (255, 100, 100)      # チャージゲージ用 (赤)
38
39 # 種類から色を取得するマッピング辞書
40 COLOR_MAP = {
41     "normal": COLOR_NORMAL,
42     "fake": COLOR_FAKE,
43     "ice": COLOR_ICE,
44     "trampoline": COLOR_TRAMPOLINE,
45     "trap": COLOR_TRAP
46 }
47
48 # =====
49 # 1. データ構造 (クラス) の定義
```

```
50 # =====
51
52 class Player:
53     def __init__(self):
54         # 画像・位置情報
55         self.image_original = pygame.image.load("9.png")
56         self.image_original = pygame.transform.scale(self.image_original, (30, 40))
57         self._image = self.image_original
58         self._rect = self._image.get_rect(center=(WIDTH // 2, HEIGHT - 100))
59
60         # 移動・物理状態
61         self._vel_x = 0
62         self._vel_y = 0
63         self._on_ground = False
64
65         # ジャンプチャージ設定
66         self._is_charging = False
67         self._charge_power = 0
68         self._max_charge = 20.0
69         self._direction = 1          # 1: 右, -1: 左
70
71         # ゲーム状態
72         self._is_clear = False
73         self._on_ice = False
74         self._is_reset = False
75
76         # クリエイティブモード設定
77         self._is_creative = False
78         self._has_cheated = False    # クリエイティブ使用履歴（記録更新不可フラグ）
79
80     # --- ゲッターメソッド ---
81     def get_image(self): return self._image
82     def get_rect(self): return self._rect
83     def get_vel_x(self): return self._vel_x
84     def get_vel_y(self): return self._vel_y
85     def get_on_ground(self): return self._on_ground
86     def get_is_charging(self): return self._is_charging
87     def get_charge_power(self): return self._charge_power
88     def get_max_charge(self): return self._max_charge
89     def get_direction(self): return self._direction
90     def get_is_clear(self): return self._is_clear
91     def get_on_ice(self): return self._on_ice
92     def get_is_reset(self): return self._is_reset
93     def get_is_creative(self): return self._is_creative
94     def get_has_cheated(self): return self._has_cheated
95
96     # --- セッターメソッド ---
97     def set_vel_x(self, value): self._vel_x = value
98     def set_vel_y(self, value): self._vel_y = value
99     def set_on_ground(self, value): self._on_ground = value
100     def set_is_charging(self, value): self._is_charging = value
101     def set_charge_power(self, value): self._charge_power = value
102     def set_direction(self, value): self._direction = value
103     def set_is_clear(self, value): self._is_clear = value
104     def set_on_ice(self, value): self._on_ice = value
105     def set_is_reset(self, value): self._is_reset = value
106     def set_is_creative(self, value): self._is_creative = value
107     def set_has_cheated(self, value): self._has_cheated = value
108
109
110 class Platform:
111     def __init__(self, x, y, w, h, color, plat_type="normal"):
112         self._image = pygame.Surface((w, h))
113         self._image.fill(color)
114         self._rect = self._image.get_rect(topleft=(x, y))
```

```
115         self._type = plat_type
116
117     # --- ゲッターメソッド ---
118     def get_image(self): return self._image
119     def get_rect(self): return self._rect
120     def get_type(self): return self._type
121
122
123     # =====
124     # 2. クラスを操作する関数（クラス外部定義）
125     # =====
126
127     def update_player(player, keys, platforms, goal_block, jump_sound):
128         """プレイヤーの移動、入力、衝突判定などの状態を更新する関数"""
129         if not player.get_is_clear():
130
131             # --- クリエイティブモードの切り替え制御 ---
132             if keys[pygame.K_LSHIFT] and keys[pygame.K_c]:
133                 player.set_is_creative(True)
134                 player.set_has_cheated(True) # チート履歴をON（トラップを踏むまで記録更新をストップ）
135                 player.set_vel_x(0)
136                 player.set_vel_y(0)
137                 player.set_is_charging(False)
138                 player.set_charge_power(0)
139             elif keys[pygame.K_LSHIFT] and keys[pygame.K_a]:
140                 player.set_is_creative(False)
141
142             # --- クリエイティブモード中の自由移動処理 ---
143             if player.get_is_creative():
144                 speed = 10
145                 rect = player.get_rect()
146
147                 if keys[pygame.K_LEFT] or keys[pygame.K_a]:
148                     rect.x -= speed
149                     player.set_direction(-1)
150                     player._image = pygame.transform.flip(player.image_original, True, False)
151                 if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
152                     rect.x += speed
153                     player.set_direction(1)
154                     player._image = player.image_original
155                 if keys[pygame.K_UP] or keys[pygame.K_w]:
156                     rect.y -= speed
157                 if keys[pygame.K_DOWN] or keys[pygame.K_s]:
158                     rect.y += speed
159
160                 # 画面左右端の移動制限
161                 if rect.left < 0: rect.left = 0
162                 if rect.right > WIDTH: rect.right = WIDTH
163
164                 return # クリエイティブ中は物理演算や通常の当たり判定をスキップ
165
166             # --- 通常モード：地上での移動とチャージ処理 ---
167             if player.get_on_ground():
168                 if player.get_on_ice():
169                     player.set_vel_x(player.get_vel_x() * 0.96) # 氷の上は徐々に減速（滑る）
170                 else:
171                     player.set_vel_x(0)
172
173             # 向きの変更
174             if keys[pygame.K_LEFT] or keys[pygame.K_a]:
175                 player.set_direction(-1)
176                 player._image = pygame.transform.flip(player.image_original, True, False)
177             if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
178                 player.set_direction(1)
179                 player._image = player.image_original
```

```
180
181     # ジャンプチャージ
182     if keys[pygame.K_SPACE]:
183         player.set_is_charging(True)
184         current_power = player.get_charge_power()
185         if current_power < player.get_max_charge():
186             player.set_charge_power(current_power + 0.4)
187     else:
188         # スペースキーが離されたらジャンプを実行
189         if player.get_is_charging():
190             power = player.get_charge_power()
191             direction = player.get_direction()
192
193             player.set_vel_y(-power * 0.9 - 5)
194             player.set_vel_x(direction * (power * 0.4 + 2))
195
196             if jump_sound:
197                 jump_sound.play()
198
199             player.set_is_charging(False)
200             player.set_charge_power(0)
201             player.set_on_ground(False)
202             player.set_on_ice(False)
203
204     # 空中にいる場合は重力を適用
205     if not player.get_on_ground():
206         player.set_vel_y(player.get_vel_y() + GRAVITY)
207
208     rect = player.get_rect()
209
210     # --- x軸方向の移動と壁・足場の側面衝突判定 ---
211     rect.x += player.get_vel_x()
212     if rect.left < 0:
213         rect.left = 0
214         player.set_vel_x(player.get_vel_x() * -0.5) # 跳ね返り
215         player.set_direction(1)
216     if rect.right > WIDTH:
217         rect.right = WIDTH
218         player.set_vel_x(player.get_vel_x() * -0.5) # 跳ね返り
219         player.set_direction(-1)
220
221     for plat in platforms:
222         if plat.get_type() == "fake":
223             continue # 紫の床 (fake) は側面判定もスルー
224
225         plat_rect = plat.get_rect()
226         if rect.colliderect(plat_rect):
227             if player.get_vel_x() > 0: # 右移動中に衝突
228                 rect.right = plat_rect.left
229                 player.set_vel_x(player.get_vel_x() * -0.5)
230                 player.set_direction(-1)
231             elif player.get_vel_x() < 0: # 左移動中に衝突
232                 rect.left = plat_rect.right
233                 player.set_vel_x(player.get_vel_x() * -0.5)
234                 player.set_direction(1)
235
236     # --- y軸方向の移動と天井・着地判定 ---
237     rect.y += int(player.get_vel_y())
238
239     # ゴールブロックとの衝突判定 (通常モード時のみクリア可能)
240     goal_rect = goal_block.get_rect()
241     if rect.colliderect(goal_rect):
242         if player.get_vel_y() < 0:
243             rect.top = goal_rect.bottom
244             player.set_vel_y(0)
```

```
245         player.set_is_clear(True)
246         pygame.mixer.music.fadeout(2000) # BGMをフェードアウト
247
248     player.set_on_ground(False)
249     player.set_on_ice(False)
250
251     # 通常の足場に対する上下の衝突判定ループ
252     for plat in platforms:
253         if plat.get_type() == "fake":
254             continue # 紫の床 (fake) は上下判定も完全にスルー
255
256         plat_rect = plat.get_rect()
257         if rect.colliderect(plat_rect):
258             # 落下中の着地判定
259             if player.get_vel_y() > 0:
260                 if rect.bottom <= plat_rect.top + player.get_vel_y() + 1:
261
262                     # トランポリン床 (オレンジ)
263                     if plat.get_type() == "trampoline":
264                         rect.bottom = plat_rect.top
265                         player.set_vel_y(-25)
266                         player.set_on_ground(False)
267                         break
268
269                     # トラップ床 (赤)
270                     if plat.get_type() == "trap":
271                         rect.center = (WIDTH // 2, HEIGHT - 100)
272                         player.set_vel_y(0)
273                         player.set_vel_x(0)
274                         player.set_is_reset(True)
275                         player.set_has_cheated(False) # スタートに戻ったらペナルティを解除
276                         break
277
278             # 通常の着地処理
279             rect.bottom = plat_rect.top
280             player.set_vel_y(0)
281
282             if plat.get_type() == "ice":
283                 player.set_on_ice(True)
284             else:
285                 player.set_on_ice(False)
286
287             player.set_on_ground(True)
288             break
289
290     # 上昇中に天井 (足場の裏側) に頭をぶつけた判定
291     elif player.get_vel_y() < 0:
292         rect.top = plat_rect.bottom
293         player.set_vel_y(0)
294         break
295
296
297     def draw_ui(screen, font, player, current_floor, total_floors, current_height, max_height):
298         """画面左上のステータスUIを描画する関数"""
299         panel_x = 10
300         panel_y = 10
301         panel_w = 300
302         panel_h = 120
303
304         # UIの枠組み
305         panel_rect = pygame.Rect(panel_x, panel_y, panel_w, panel_h)
306         pygame.draw.rect(screen, DARK_PANEL, panel_rect, border_radius=12)
307         pygame.draw.rect(screen, PANEL_BORDER, panel_rect, 2, border_radius=12)
308
309         # テキストレンダリングと配置
```



```
310 title_text = font.render("STATUS", True, GOLD)
311 screen.blit(title_text, (panel_x + 15, panel_y + 10))
312
313 floor_text = font.render(f"Floor {current_floor} / {total_floors}", True, WHITE)
314 screen.blit(floor_text, (panel_x + 15, panel_y + 42))
315
316 height_text = font.render(f"Height: {current_height // 10} m", True, LIGHT_BLUE)
317 screen.blit(height_text, (panel_x + 15, panel_y + 70))
318
319 # クリエイティブ使用中はベスト記録の色を変更（更新停止中を表現）
320 max_color = ORANGE if not player.get_has_cheated() else (150, 150, 150)
321 max_text = font.render(f"Best: {max_height // 10} m", True, max_color)
322 screen.blit(max_text, (panel_x + 180, panel_y + 70))
323
324 # クリエイティブモード稼働中のテキスト表示
325 if player.get_is_creative():
326     creative_text = font.render("[ CREATIVE MODE ]", True, (255, 100, 255))
327     screen.blit(creative_text, (panel_x + 5, panel_y + panel_h + 5))
328
329
330 # =====
331 # 3. メインループ
332 # =====
333
334 ✓ def main():
335     pygame.init()
336     pygame.mixer.init()
337
338     screen = pygame.display.set_mode((WIDTH, HEIGHT))
339     pygame.display.set_caption("Jump King - Colored Gimmicks")
340     clock = pygame.time.Clock()
341     font = pygame.font.SysFont(None, 36)
342     large_font = pygame.font.SysFont(None, 72)
343
344     # サウンドの読み込み
345     jump_sound = None
346     if os.path.exists("jump.wav"):
347         jump_sound = pygame.mixer.Sound("jump.wav")
348         jump_sound.set_volume(0.4)
349
350     if os.path.exists("BGM.mp3"):
351         pygame.mixer.music.load("BGM.mp3")
352         pygame.mixer.music.set_volume(0.25)
353         pygame.mixer.music.play(-1)
354
355     player = Player()
356
357     # 背景画像の読み込みとサイズ調整
358     bg_large = pygame.image.load("background_all.png")
359     bg_large = pygame.transform.scale(bg_large, (600, 2400))
360
361     TOTAL_FLOORS = 10
362     platforms = []
363
364     # 初期位置の床（一番下）
365     platforms.append(Platform(0, HEIGHT - 40, WIDTH, 40, COLOR_NORMAL, "normal"))
366
367     # 固定ステージデータ（下から順にすべての床の番号をコメントアウトで記載）
368     # 形式: (x座標, y座標, 幅, ギミック種類)
369     fixed_level_data = [
370         # Floor 1 (Easy - 広くて近い)
371         (100, 600, 150, "normal"), # 床1
372         (400, 450, 150, "normal"), # 床2
373         (150, 300, 150, "normal"), # 床3
374         (350, 150, 150, "normal"), # 床4
```

```
375         (250, 0, 150, "normal"), # 床5
376
377     # Floor 2 (Easy - 少し狭くなる)
378     (50, -150, 120, "normal"), # 床6
379     (400, -300, 120, "normal"), # 床7
380     (100, -450, 120, "normal"), # 床8
381     (350, -600, 120, "normal"), # 床9
382     (200, -750, 120, "normal"), # 床10
383
384     # Floor 3 (氷の床 初登場)
385     (450, -900, 100, "normal"), # 床11
386     (150, -1050, 100, "ice"), # 床12
387     (400, -1200, 100, "normal"), # 床13
388     (200, -1350, 100, "ice"), # 床14
389     (100, -1500, 100, "normal"), # 床15
390
391     # Floor 4 (トランポリン 初登場)
392     (350, -1650, 100, "trampoline"), # 床16
393     (100, -2100, 120, "normal"), # 床17
394     (400, -2250, 100, "normal"), # 床18
395     (200, -2400, 100, "normal"), # 床19
396
397     # Floor 5 (すり抜ける罠 初登場)
398     (50, -2550, 100, "normal"), # 床20
399     (300, -2600, 100, "fake"), # 床21
400     (400, -2700, 100, "normal"), # 床22
401     (150, -2850, 100, "ice"), # 床23
402     (50, -3000, 100, "fake"), # 床24
403     (350, -3050, 100, "normal"), # 床25
404     (100, -3200, 100, "normal"), # 床26
405
406     # Floor 6 (幅が狭くなる)
407     (450, -3350, 80, "normal"), # 床27
408     (200, -3500, 80, "ice"), # 床28
409     (50, -3650, 80, "trampoline"), # 床29
410     (350, -4100, 80, "normal"), # 床30
411
412     # Floor 7 (トラップ床 初登場)
413     (150, -4250, 80, "normal"), # 床31
414     (400, -4400, 80, "trap"), # 床32
415     (450, -4500, 80, "normal"), # 床33
416     (100, -4650, 80, "normal"), # 床34
417     (250, -4800, 80, "ice"), # 床35
418
419     # Floor 8 (複合ギミック)
420     (450, -4950, 70, "normal"), # 床36
421     (50, -5100, 70, "fake"), # 床37
422     (150, -5200, 70, "normal"), # 床38
423     (300, -5350, 70, "trap"), # 床39
424     (50, -5450, 70, "normal"), # 床40
425     (400, -5600, 70, "trampoline"), # 床41
426
427     # 41番目と42番目の間の追加床 (左寄り)
428     (80, -5850, 70, "normal"), # 床42
429
430     # Floor 9 (終盤・かなり狭い)
431     (75, -6050, 70, "normal"), # 床43
432     (400, -6200, 70, "ice"), # 床44
433     (150, -6350, 70, "trap"), # 床45
434     (200, -6450, 70, "normal"), # 床46
435
436     # Floor 10 (最後の関門)
437     (450, -6600, 60, "normal"), # 床47
438     (250, -6750, 60, "ice"), # 床48
439     (50, -6900, 60, "trampoline"), # 床49
```

```
440 ]
441
442 # 固定ステージデータを読み込んでリスト化
443 for data in fixed_level_data:
444     x, y, w, plat_type = data
445     platforms.append(Platform(x, y, w, 20, COLOR_MAP[plat_type], plat_type))
446
447 # ゴールブロックの設定
448 goal_y = (HEIGHT - 40) - (HEIGHT * TOTAL_FLOORS)
449 goal_block = Platform(0, goal_y, WIDTH, 40, GOLD, "goal")
450
451 camera_y = 0
452 max_height = 0
453 clear_time = None # クリアした瞬間のゲームティック記録用
454
455 running = True
456 while running:
457     for event in pygame.event.get():
458         if event.type == pygame.QUIT:
459             running = False
460
461     keys = pygame.key.get_pressed()
462
463     # プレイヤー状態のアップデート
464     update_player(player, keys, platforms, goal_block, jump_sound)
465
466     p_rect = player.get_rect()
467
468     # 赤いトラップ床を踏んでリセットされた場合のカメラ同期
469     if player.get_is_reset():
470         camera_y = 0
471         player.set_is_reset(False)
472
473     # カメラの上方向スクロール処理
474     SCROLL_TOP_MARGIN = 200
475     if p_rect.y - camera_y < SCROLL_TOP_MARGIN:
476         camera_y = p_rect.y - SCROLL_TOP_MARGIN
477
478     # カメラの下方向滑らかな追従スクロール処理
479     SCROLL_BOTTOM_MARGIN = 150
480     camera_easing_down = 0.08
481     if p_rect.bottom - camera_y > HEIGHT - SCROLL_BOTTOM_MARGIN:
482         target_camera_y = p_rect.bottom - (HEIGHT - SCROLL_BOTTOM_MARGIN)
483         camera_y += (target_camera_y - camera_y) * camera_easing_down
484
485     if camera_y > 0:
486         camera_y = 0
487
488     # 高さ計算とベスト記録の保存（クリエイティブ使用中は記録更新をブロック）
489     current_height = (HEIGHT - 40) - p_rect.bottom
490     if current_height > max_height and not player.get_has_cheated():
491         max_height = current_height
492
493     current_floor = min((current_height // HEIGHT) + 1, TOTAL_FLOORS)
494     display_floor = max(1, current_floor)
495
496     # 背景描画（パララックス効果付き）
497     bg_y = 1600 + (camera_y * 0.2)
498     if bg_y < 0: bg_y = 0
499     src_rect = pygame.Rect(0, int(bg_y), 600, 800)
500     screen.blit(bg_large, (0, 0), src_rect)
501
502     # ゴールブロックの描画
503     goal_rect = goal_block.get_rect()
504     goal_draw_y = goal_rect.y - camera_y
```

```
505     if -100 < goal_draw_y < HEIGHT + 100:
506         screen.blit(goal_block.get_image(), (goal_rect.x, goal_draw_y))
507
508     # 足場の描画
509     for plat in platforms:
510         plat_rect = plat.get_rect()
511         draw_y = plat_rect.y - camera_y
512         if -50 < draw_y < HEIGHT + 50:
513             screen.blit(plat.get_image(), (plat_rect.x, draw_y))
514
515     # プレイヤーの描画
516     player_draw_y = p_rect.y - camera_y
517     screen.blit(player.get_image(), (p_rect.x, player_draw_y))
518
519     # 通常モードかつジャンプチャージ中のみゲージを描画
520     if player.get_is_charging() and not player.get_is_clear() and not player.get_is_creative():
521         gauge_width = (player.get_charge_power() / player.get_max_charge()) * 40
522         pygame.draw.rect(screen, COLOR_RED_UI, (p_rect.centerx - 20, player_draw_y - 15, gauge_width, 8))
523
524     # プレイヤーの向きを示すガイドドットの描画
525     arrow_x = p_rect.right + 5 if player.get_direction() == 1 else p_rect.left - 10
526     pygame.draw.rect(screen, WHITE, (arrow_x, player_draw_y + 15, 5, 5))
527
528     # 情報表示 (UIの描画)
529     draw_ui(screen, font, player, current_floor, TOTAL_FLOORS, current_height, max_height)
530
531     # クリア時の演出および10秒後の自動シャットダウン処理
532     if player.get_is_clear():
533         if clear_time is None:
534             clear_time = pygame.time.get_ticks()
535
536         # クリアテキスト描画
537         clear_text = large_font.render("CLEAR!!", True, GOLD)
538         text_rect = clear_text.get_rect(center=(WIDTH // 2, HEIGHT // 2 - 50))
539         screen.blit(clear_text, text_rect)
540
541         sub_text = font.render("CONGRATULATIONS!", True, WHITE)
542         sub_rect = sub_text.get_rect(center=(WIDTH // 2, HEIGHT // 2 + 10))
543         screen.blit(sub_text, sub_rect)
544
545         # 自動終了までの秒数カウントダウン表示
546         elapsed = pygame.time.get_ticks() - clear_time
547         remaining = max(0, 10 - (elapsed // 1000))
548         close_text = font.render(f"Closing in {remaining} seconds...", True, LIGHT_BLUE)
549         close_rect = close_text.get_rect(center=(WIDTH // 2, HEIGHT // 2 + 60))
550         screen.blit(close_text, close_rect)
551
552         # 10秒 (10000ミリ秒) 経過でループを終了
553         if elapsed >= 10000:
554             running = False
555
556     pygame.display.flip()
557     clock.tick(FPS)
558
559     pygame.quit()
560     sys.exit()
561
562 if __name__ == "__main__":
563     main()
```